

- Use `HashMap<String, List<Employee>>` for departmental storage
- Use Iterator to traverse both the map entries and the lists
- Use Iterator for safe removal while iterating
- Create methods to display department-wise statistics

Your Code

```

1 import java.util.*;
2 public class EmployeePayroll {
3     public static void main(String[] args) {
4         HashMap<String, ArrayList<Employee>> map = new HashMap<>();
5         map.put("IT", new ArrayList<Employee>());
6         map.put("HR", new ArrayList<Employee>());
7         map.get("IT").add(new Employee(101, "Ravi", "IT", 50000));
8         map.get("IT").add(new Employee(102, "Arun", "IT", 60000));
9         map.get("HR").add(new Employee(201, "Priya", "HR", 45000));
10        map.get("HR").add(new Employee(202, "Meena", "HR", 55000));
11        Iterator<String> dept = map.keySet().iterator();
12        while (dept.hasNext()) {
13            String d = dept.next();
14            System.out.println("\nDepartment : " + d);
15            ArrayList<Employee> list = map.get(d);
16            int total = 0;
17            Employee highest = list.get(0);
18            Iterator<Employee> it = list.iterator();
19            while (it.hasNext()) {
20                Employee e = it.next();
21                System.out.println(e.id + " " + e.name + " " + e.salary);
22                total = total + e.salary;
23                if (e.salary > highest.salary) {
24                    highest = e;
25                }
26            }
27            System.out.println("Total Salary = " + total);
28            System.out.println("Highest Paid = " + highest.name + " ");
29        }
30        Employee temp = map.get("IT").get(0);
31        map.get("HR").add(temp);
32        System.out.println("\nEmployee Moved to HR");
33        System.out.println("\nHR Department");
34        Iterator<Employee> hr = map.get("HR").iterator();
35        while (hr.hasNext()) {
36            Employee e = hr.next();
37            System.out.println(e.id + " " + e.name + " " + e.salary);
38        }
39    }
40 }
41 class Employee {
42     int id;
43     String name;
44     String department;
45     int salary;
46     Employee(int id, String name, String department, int salary) {
47         this.id = id;
48         this.name = name;
49         this.department = department;
50         this.salary = salary;
51     }
52 }

```

Input

stdin input.

Output

```

Department : HR
201 Priya 45000
202 Meena 55000
Total Salary = 100000
Highest Paid = Meena 55000

```

7 mark

4/4 answered

• Use Stack<String> for forward history

• Use Iterator to display all pages in both stacks without modifying them

• Handle edge cases (empty stack, end of navigation)

Your Code

```

1 import java.util.*;
2 public class NavigationHistory {
3     public static void main(String[] args) {
4         Browser browser = new Browser();
5         browser.visitPage("google.com");
6         browser.visitPage("youtube.com");
7         browser.visitPage("gmail.com");
8         browser.goBack();
9         browser.goBack();
10        browser.goForward();
11        browser.display();
12    }
13 }
14 class Browser {
15     Stack<String> back = new Stack<>();
16     Stack<String> forward = new Stack<>();
17     String current = "";
18     void visitPage(String url) {
19         if (!current.equals("")) {
20             back.push(current);
21         }
22         current = url;
23         forward.clear();
24     }
25     void goBack() {
26         if (!back.empty()) {
27             forward.push(current);
28             current = back.pop();
29         } else {
30             System.out.println("No previous page");
31         }
32     }
33     void goForward() {
34         if (!forward.empty()) {
35             back.push(current);
36             current = forward.pop();
37         } else {
38             System.out.println("No next page");
39         }
40     }
41     void display() {
42         System.out.println("Current Page: " + current);
43         System.out.println("\nBack History:");
44         Iterator<String> it1 = back.iterator();
45         while (it1.hasNext()) {
46             System.out.println(it1.next());
47         }
48         System.out.println("\nForward History:");
49         Iterator<String> it2 = forward.iterator();
50         while (it2.hasNext()) {
51             System.out.println(it2.next());
52         }
53     }
54 }

```

Input

stdin input

Output

```

Back History:
google.com

Forward History:
gmail.com

```

```

1 import java.util.*;
2
3 public class StudentGradeAnalyzer {
4     public static void main(String[] args) {
5         ArrayList<Student> list = new ArrayList<>();
6         Student s1 = new Student(101, "Ravi");
7         s1.grades.put("Math", 80);
8         s1.grades.put("Science", 75);
9         s1.grades.put("English", 90);
10        list.add(s1);
11        Student s2 = new Student(102, "Priya");
12        s2.grades.put("Math", 60);
13        s2.grades.put("Science", 70);
14        s2.grades.put("English", 65);
15        list.add(s2);
16        Student s3 = new Student(103, "Arun");
17        s3.grades.put("Math", 95);
18        s3.grades.put("Science", 90);
19        s3.grades.put("English", 85);
20        list.add(s3);
21        for (Student s : list) {
22            int total = 0;
23            for (int mark : s.grades.values()) {
24                total = total + mark;
25            }
26            s.average = total / s.grades.size();
27        }
28        Collections.sort(list, new Comparator<Student>() {
29            public int compare(Student a, Student b) {
30                return b.average - a.average;
31            }
32        });
33        System.out.println("Top Students");
34        for (int i = 0; i < list.size() && i < 3; i++) {
35            Student s = list.get(i);
36            System.out.println(s.id + " " + s.name + " " + s.average);
37        }
38        System.out.println("\nAll Students");
39        Iterator<Student> it = list.iterator();
40        while (it.hasNext()) {
41            Student s = it.next();
42            System.out.println(s.id + " " + s.name + " " + s.average);
43        }
44    }
45 }
46 class Student {
47
48     int id;
49     String name;
50     HashMap<String, Integer> grades = new HashMap<>();
51     int average;
52
53     Student(int id, String name) {
54         this.id = id;
55         this.name = name;
56     }
57 }

```

Input

stdin input

Output

```

Top Students
103 Arun 90
101 Ravi 81
102 Priya 65

```

```

All Students

```

10 marks

Q2

Student Grade Analyzer
1 mark



Q3

Navigation History (Web Browser)
1 mark



Q4

Employee Payroll System
1 mark



4/4 answered

You are developing a Library Management System where books are stored in a HashSet to ensure no duplicate books (based on ISBN). The system needs to:

1. Add books with attributes: ISBN, title, author, and publication year
2. Remove a book by its ISBN
3. Display all books sorted by publication year (newest first)
4. Find all books by a specific author

Requirements:

- Use HashSet<Book> for storing books
- Implement Iterator to traverse and remove books
- Create a custom Comparator for sorting
- Display books using iterator

Your Code

```
1 import java.util.*;
2 public class LibraryManagement {
3     public static void main(String[] args) {
4         HashSet<Book> books = new HashSet<>();
5         books.add(new Book("101", "Java", "James", 2020));
6         books.add(new Book("102", "Python", "Guido", 2022));
7         books.add(new Book("103", "C++", "Bjarne", 2018));
8         books.add(new Book("104", "Data Structures", "James", 2021));
9         ArrayList<Book> list = new ArrayList<>(books);
10        Collections.sort(list, new Comparator<Book>() {
11            public int compare(Book a, Book b) {
12                return b.year - a.year;
13            }
14        });
15        System.out.println("Books Sorted by Year");
16        Iterator<Book> it = list.iterator();
17        while (it.hasNext()) {
18            Book b = it.next();
19            System.out.println(b.isbn + " " + b.title + " " + b.author);
20        }
21        String authorName = "James";
22        System.out.println("\nBooks by " + authorName);
23        Iterator<Book> it2 = list.iterator();
24        while (it2.hasNext()) {
25            Book b = it2.next();
26            if (b.author.equals(authorName)) {
27                System.out.println(b.isbn + " " + b.title + " " + b.year);
28            }
29        }
30    }
31 }
32 class Book {
33     String isbn;
34     String title;
35     String author;
36     int year;
37     Book(String isbn, String title, String author, int year) {
38         this.isbn = isbn;
39         this.title = title;
40         this.author = author;
41         this.year = year;
42     }
43 }
44 }
```

Input

stdin input

Output

```
Books Sorted by Year
102 Python Guido 2022
104 Data Structures James 2021
101 Java James 2020
103 C++ Bjarne 2018
```